CSEN 296B-2 | Week 2.2 Lab Submission

Backlog and Sprint Zero Plan for AstraNotes

Student Name: Alex Shirazi

Date: April 10, 2026

Project: AstraNotes

Chosen Technical Path: Python

1. User Stories and Acceptance Criteria

The following eight user stories are derived directly from the Requirement Set established in Week 1.2 (FR-01 through FR-08, SPR-01 through SPR-04). Each story includes testable acceptance criteria aligned with the Definition of Done from Week 2.1.

US-01 — Create a Note

Requirement: FR-01

As a user, I want to create a new note with a title and body so that I can capture information quickly and find it later.

Acceptance Criteria:

- Given I trigger "New Note," a form is presented with a title field and a body field.
- When I submit a valid title and body, a Note is created with a UUID, created_at, and modified_at set automatically.
- The new note appears in the note list immediately, sorted by modified_at descending.
- Submitting an empty title is rejected with a non-technical error message (ValidationError).

US-02 — Edit an Existing Note

Requirement: FR-02

As a user, I want to edit the title or body of an existing note so that I can keep my notes accurate and up to date.

Acceptance Criteria:

- When I modify and save a note, the content is updated in storage and modified_at reflects the save time.
- The original created_at timestamp is unchanged by an edit.

- If the save operation fails, a clear error message is shown and the note is not corrupted.
- The edited note appears at the top of the note list after saving.

US-03 — Delete a Note

Requirement: FR-03

As a user, I want to delete a note I no longer need so that my note collection stays organized.

Acceptance Criteria:

- When I trigger delete and confirm, the note is removed from storage and no longer appears in the list.
- If I cancel the delete confirmation, the note is not removed.
- Attempting to delete a note that does not exist surfaces a clear error rather than crashing.

US-04 — Mark a Note as Private

Requirement: FR-04, SPR-01

As a user, I want to mark a note as private so that its content is encrypted on disk and not readable without authorization.

Acceptance Criteria:

- When is_private is set to True and the note is saved, the body is encrypted by PrivacyService before being written to disk.
- When a private note is retrieved, the body is decrypted by PrivacyService before display.
- The raw note file on disk contains no plaintext body content for any private note.
- If encryption or decryption fails, a clear error is shown and no partial content is displayed.

US-05 — Persist Notes Across Sessions

Requirement: FR-05, NFR-03

As a user, I want my notes to be saved to local storage so that they are available the next time I open the application.

Acceptance Criteria:

- On startup, all previously saved notes are loaded from local file storage and displayed.
- Notes created or edited in the current session are available in the next session without additional action.

- If a note file is missing or corrupt on startup, the app logs the issue and continues loading remaining notes.
- Startup completes and the note list is displayed within 3 seconds on a standard development machine.

US-06 — Search Notes by Keyword

Requirement: FR-07, NFR-01

As a user, I want to search my notes by keyword so that I can find relevant notes without scrolling the full list.

Acceptance Criteria:

- Entering a keyword returns all notes whose title or body contains it (case-insensitive).
- If no notes match, a clear empty-state message is shown.
- Search across up to 500 notes completes within 2 seconds.
- Clearing the search field restores the full sorted note list.

US-07 — View Notes Sorted by Last Modified

Requirement: FR-08

As a user, I want to see my notes listed with the most recently modified note at the top so that my most active notes are easy to reach.

Acceptance Criteria:

- The note list is sorted by modified_at descending by default on every application load.
- After creating or editing a note, it appears at the top of the list without a manual refresh.
- Notes with identical modified_at values are ordered consistently using created_at as a tiebreaker.

US-08 — Tag a Note

Requirement: FR-06

As a user, I want to add tags to a note so that I can group related notes by topic.

Acceptance Criteria:

- Tags are persisted as a list of strings as part of the note metadata.
- Tags are preserved exactly as entered (case-sensitive).
- A note with no tags displays an empty tag list without error.
- Tags can be added, edited, or removed in the same edit flow as title and body.

---

2. Prioritized Backlog

Stories are ordered by a Value/Risk criterion: stories that deliver core user value or de-risk the architecture are placed first. Stories that depend on a stable core follow.

Pri	Story	Requirement(s)	Rationale
1	US-05 — Persist Notes Across Sessions	FR-05, NFR-03	Foundation: nothing works if notes don't survive sessions.
2	US-01 — Create a Note	FR-01, FR-06	Primary user action. Validates Note model, NoteRepository, and ValidationLayer end-to-end.
3	US-04 — Mark a Note as Private	FR-04, SPR-01	Highest architectural risk. PrivacyService must be verified before the save/load path is stable.
4	US-02 — Edit an Existing Note	FR-02	Depends on create and persist. Validates the update path through NoteRepository.
5	US-03 — Delete a Note	FR-03	Depends on create and persist. Validates the delete path and error handling.
6	US-07 — View Notes Sorted by Last Modified	FR-08	Low implementation risk. Confirms sort logic on the list view.
7	US-06 — Search Notes by Keyword	FR-07, NFR-01	Depends on a populated note collection. Performance requirement needs real data to validate.
8	US-08 — Tag a Note	FR-06	Metadata enhancement. Depends on a stable note model and edit flow. Lowest priority.

Why US-05 before US-01: The persistence layer validates the entire storage architecture from Submission 1. Building it first means all subsequent stories test against a real storage backend rather than a stub.

Why US-04 third: The PrivacyService is the highest architectural risk. Addressing it early prevents having to re-thread the full CRUD loop through encryption later.

US-08 last: Tags are a metadata enhancement that add value but don't change core architecture. Parked as a Sprint 1 or Sprint 2 candidate depending on progress.

3. Sprint Zero Plan

Sprint Duration: Week 2.2 through end of Week 3 | **Goal:** Establish the project foundation so that Sprint 1 can begin implementation with zero ambiguity about structure, tools, workflow, or architecture.

Sprint Zero is not a feature sprint. No user stories are marked Done at the end of Sprint Zero. The sprint is complete when the working environment, architecture skeleton, and planning artifacts are all in place and verified against the Definition of Done.

Objective 1: Repository and Project Structure **(FR-05, SPR-04)**

- Initialize a Python project repository with folders: astranotes/ (models, repository, privacy, validation, cli), tests/, planning/, docs/.
- Add requirements.txt listing only verified, licensed dependencies.
- Add a .gitignore appropriate for Python projects.
- Confirm the project runs with python -m astranotes without errors (placeholder CLI acceptable).
- DoD check: structure matches architecture from Submission 1, no untracked dependencies, project runs cleanly.

Objective 2: Note Data Model **(FR-01, FR-06)**

- Implement the Note dataclass: id (UUID), title (str), body (str), is_private (bool), created_at (datetime), modified_at (datetime), tags (list of str).
- Write unit tests: Note instantiates with valid fields; created_at and modified_at are set automatically; id is a valid UUID.
- DoD check: model is explainable, tested, and traced to FR-01 and FR-06.

Objective 3: NoteRepository Interface and JsonFileRepository **(FR-05)**

- Implement abstract NoteRepository with method signatures: save, get, list_all, update, delete.
- Implement JsonFileRepository that reads/writes individual .json files per note to a local data directory.
- Write unit test: note can be saved to a temp directory and retrieved with matching fields.
- DoD check: repository interface in place; save/load round trip passes; traced to FR-05.

Objective 4: PrivacyService Skeleton **(FR-04, SPR-01)**

- Implement PrivacyService with encrypt(body: str) -> bytes and decrypt(data: bytes) -> str using Fernet (cryptography library).
- Write unit test: encrypting a string and decrypting it returns the original value; encrypted output is not equal to plaintext.
- Risk note: Key management is deferred to Sprint 1. A hardcoded test key is acceptable in the test suite only.

• DoD check: PrivacyService tested for basic round-trip; no plaintext stored for private notes; traced to SPR-01 and FR-04.

Objective 5: ValidationLayer Skeleton **(FR-01)**

• Implement ValidationLayer with at least one rule: note title must be a non-empty string.

• Raise a custom ValidationError when the rule is violated.

• Write unit test: empty title raises ValidationError; valid title passes.

• DoD check: ValidationLayer tested and traced to FR-01.

Objective 6: Workflow Readiness Check **(All)**

• Confirm ai_log.md and backlog.md are in place and reflect Sprint Zero work.

• Confirm the Architecture Decision Log (Submission 1) is accessible and up to date.

• Confirm the Working Agreement and Definition of Done (Submission 2.1) are applied to every Sprint Zero task.

• Run all unit tests and confirm 100% pass rate before Sprint Zero is closed.

Sprint Zero Risks

Risk	Likelihood	Mitigation
Fernet key management unclear for production	Medium	Defer to Sprint 1; use test key only in Sprint Zero. Document in ADL.
JSON edge cases (corrupt files, missing notes)	Low	Implement basic error handling in Sprint Zero. Defer advanced recovery.
Sprint Zero expands into feature work	Medium	No user story is marked Done in Sprint Zero. Feature work is added to backlog.
Architecture needs adjustment once code is written	Low–Medium	Sprint Zero skeleton is the test. Update ADL before Sprint 1 if changes are needed.

Sprint Zero Exit Criteria

Sprint Zero is complete only when all of the following are true:

- ■ `Project repository is initialized with the folder structure above.`
- ■ `Note dataclass is implemented and has passing unit tests.`
- ■ `NoteRepository interface and JsonFileRepository are implemented with a passing save/load test.`
- ■ `PrivacyService is implemented with a passing encrypt/decrypt test.`
- ■ `ValidationLayer is implemented with a passing title validation test.`
- ■ `ai_log.md and backlog.md are populated and current.`
- ■ `All tests pass.`

■ No user story from the backlog is prematurely marked Done.

4. How AI Helped and What I Refined

AI was used as a planning partner across three phases of this lab: drafting, challenging, and refining.

What AI helped with:

• Generating an initial set of user stories from the requirement IDs in Submission 2. The first draft covered the core FR items but missed SPR-01 as a constraint on US-04 (private note encryption). I added that link manually after reviewing the requirement set.

• Drafting the backlog priority order. The AI initially ranked search (US-06) above persistence (US-05), which is a common pattern for feature-driven planning but wrong for this architecture. I re-ordered based on the dependency structure in the architecture from Submission 1.

• Generating the Sprint Zero objective list. The AI's first draft included a "UI design" objective that was not appropriate for a CLI-first Python project. I removed it and replaced it with the ValidationLayer skeleton, which is a real architectural dependency.

What I rejected:

• A suggested story for "user authentication" that is entirely out of scope for a local single-user note app.

• A Sprint Zero objective for "CI/CD pipeline setup" that is premature for a solo quarter-length project.

• Acceptance criteria phrased as "the system should..." — changed all criteria to specific, testable conditions using Given/When/Then framing.

The process confirmed the Working Agreement from Week 2.1: AI is useful for generating a first draft quickly, but every output required review against the requirement set, the architecture, and the project scope before it was accepted. The final artifacts reflect my judgment, not the AI's first answer.